Relazione progetto - Parole

Indice

1. Introduzione e panoramica del progetto
2. Formalizzazione del problema
3. Identificazione delle problematiche incontrate nella soluzione proposta
4. Descrizione della strategia e delle procedure adottate
5. Algoritmi e procedure alternative
6. Casi di test significativi e casi limite

1 Introduzione – Panoramica del progetto

Il progetto **Parole** realizza un dizionario dinamico di parole e schemi in Go, pensato per eseguire tutte le operazioni richieste dalla traccia d'esame (carica, inserisci, elimina, stampa, ricerca di schemi, calcolo della distanza di editing e gestione delle catene). L'obiettivo principale è fornire una soluzione in grado di gestire l'aggiornamento continuo dei dati e di rispondere alle diverse interrogazioni richieste dal programma.

Per raggiungere questi risultati, sono state scelte e sviluppate, trovando buoni compromessi per quanto riguarda complessità spaziale e temporale, strutture dati e algoritmi che permettono di bilanciare sia la rapidità delle operazioni che la semplicità della gestione complessiva del dizionario.

2 Formalizzazione del problema

Il problema è stato modellato con due strutture principali:

- Struttura **Dizionario**, contente un doppio indice complementare:
  - **sequenze**: definita come *map[byte][]tipoSequenza*, è una mappa che utilizza come chiavi le lettere (a–z) e contiene sia parole che schemi. In questo modo, le operazioni di inserimento, eliminazione e controllo dei duplicati vengono eseguite solo sul sottoinsieme di elementi che condividono la stessa lettera iniziale, riducendo sensibilmente il costo computazionale.
  - **indice**: definita come *map[int][]string*, è una mappa che contiene tutte le parole suddivise in base alla loro lunghezza. Questa struttura permette di generare rapidamente i vicini (a distanza 1) e di eseguire una BFS per le catene senza dover scandire l'intero dizionario lavorando su una porzione ridotta dell'intera struttura.
- Struttura **tipoSequenza,** che rappresenta ogni singolo elemento del dizionario (sia parola sia schema) contente:
  - **Sequanza**: definita come *string*, è il testo della sequenza inserita.
  - **isWord**: variabile booleana che permette di capire se la sequenza inserita è una parola o uno schema.

Ogni elemento della struttura *sequenze* è rappresentato da un tipo personalizzato che contiene sia il testo della sequenza sia una variabile booleana (isWord), che consente al programma di distinguere in modo semplice e immediato parole e schemi, senza la necessità di strutture dati aggiuntive. Questo approccio rende la gestione del dizionario più ordinata ed efficiente, semplificando sia le operazioni di aggiornamento che le varie ricerche richieste dal progetto. In particolare, la suddivisione per iniziale e per lunghezza permette di eseguire rapidamente tutte le operazioni chiave, come la verifica di compatibilità tra parola e schema, il calcolo della distanza di editing e la costruzione delle catene di parole.

Principali algoritmi su cui si basa la soluzione proposta:

- **Compatibilità** (schema–parola): la verifica tra schema e parola avviene con un singolo passaggio sulla parola, controllando sia le lettere fisse sia le eventuali ripetizioni di maiuscole nello schema.

- **Distanza di editing**: si basa sull'algoritmo di Damerau-Levenshtein, che calcola la distanza minima tra due parole considerando anche le trasposizioni di caratteri adiacenti come un'unica operazione (con costo 1). Questo garantisce coerenza tra il calcolo della distanza e la costruzione del grafo utilizzato per la ricerca delle catene.
- **Catena minima**: la ricerca della catena minima tra due parole si basa su una visita in ampiezza (BFS), che permette di trovare il percorso più breve nel grafo delle parole collegate da una distanza di editing.

La scelta di utilizzare due indici (per iniziale e per lunghezza) consente di avere un bilanciamento tra efficienza nell'aggiornamento e rapidità nelle interrogazioni, modellando il problema in modo conforme alle richieste della traccia ma mantenendo una complessità contenuta.

3 Problematiche della soluzione proposta

La scelta di utilizzare due strutture dati principali (una mappa per iniziale e una mappa per lunghezza) introduce sicuramente dei vantaggi in termini di efficienza delle operazioni più comuni, ma comporta anche alcune criticità e svantaggi sia dal punto di vista dello spazio occupato sia dal punto di vista della gestione e del tempo di esecuzione.

Ridondanza dei dati e complessità spaziale elevata

Uno degli aspetti principali è la ridondanza: ogni parola viene memorizzata in entrambe le strutture (sequenze e indice). Questo significa che, se il dizionario cresce molto, la memoria occupata aumenta più rapidamente rispetto a una soluzione in cui i dati sono memorizzati una sola volta. In particolare:

- Ogni parola viene salvata almeno due volte (una come tipoSequenza, una come stringa nell'indice).
- Il tempo di aggiornamento raddoppia poiché bisogna aggiornare la doppia struttura. Quindi, il tempo totale può aumentare rispetto a una struttura unica.
- Il consumo di memoria può diventare significativo per dizionari di grandi dimensioni.

Performance nelle operazioni di ricerca e aggiornamento

Se da un lato la suddivisione per iniziale e per lunghezza accelera le ricerche più frequenti, dall'altro bisogna considerare che:

- Le operazioni di aggiornamento sono più lente, perché ogni inserimento/cancellazione comporta due accessi e due modifiche.
- Se il numero di parole associate a una certa iniziale o a una certa lunghezza è molto alto, alcune operazioni potrebbero comunque richiedere scansioni di molteplici elementi, riducendo l'efficacia dell'ottimizzazione.

Scalabilità limitata

In scenari con dizionari molto grandi (milioni di parole), la duplicazione delle informazioni può rendere la soluzione meno scalabile rispetto ad altre strutture.

4 Strategia e funzioni sviluppate

Le funzioni main, newDizionario, crea, esegui, carica, inserisci, elimina, stampa_parole e stampa_schemi sono state sviluppate come da richiesta dalla traccia.

Complessità di ciascuna di queste funzioni:

	Tempo	Spazio
NewDizionario()	O(1)	O(1)
Crea()	O(1)	O(1)
Esegui()	O(m) con *m* lunghezza della riga, più il costo della singola operazione instradata	O(1)
Carica()	O(T x (L +K)), con T elementi letti dal file	O(1)
Inserisci()	O(L + k), L lunghezza della sequenza, k elementi ispezionati	O(1)
Elimina()	O(k + p) elementi ispezionati per una certa lettera indice, p numero di parole con la stessa lunghezza della parola eliminata	O(1)
stampa_parole() & stampa_schemi()	O(n), n numero di elementi da stampare	O(1)

- **func creaChiaveMinuscola(c rune) byte**
  Prende in input un carattere e restituisce la corrispondente lettera minuscola. È particolarmente utile quando si lavora con schemi che iniziano con una lettera maiuscola, assicurando così che tutte le chiavi utilizzate nella mappa sequenze siano uniformi e in minuscolo. In questo modo, parole e schemi che iniziano con la stessa lettera, indipendentemente dal fatto che sia maiuscola o minuscola, vengono correttamente raggruppati sotto la stessa chiave nella mappa. La complessità spaziale e temporale è O(1).

- **func checkIniziale(w string) (bool, byte)**
  Analizza la sequenza ricevuta in input per determinare se si tratta di uno schema o di una parola normale. Se nella sequenza è presente almeno una lettera maiuscola, la funzione la identifica come uno *schema* e se la prima lettera della sequanze è maiuscola la utilizzerà (convertita in minuscolo tramite l'invocazione di creaChiaveMinuscola) come chiave per la mappa. In caso contrario, considera la sequenza come una *parola* e usa la prima lettera (già minuscola) come chiave. Restituisce quindi un valore booleano che indica se si tratta di uno schema e la chiave corrispondente.

  Complessità:

  - Tempo: O(L) con L lunghezza della stringa passata
  - Spazio: O(1)

- **func (d *dizionario) checkDuplicati(w string, iniziale byte) bool**
  Controlla se la sequenza w è già presente nel dizionario. Scorre solamente la lista delle sequenze associate alla lettera specificata dal parametro iniziale nella mappa sequenze e verifica se esiste già una sequenza identica a quella passata in input. Restituisce true se trova un duplicato, altrimenti false.

  Complessità:

  - Tempo: O(k) con k quantità ti parole sotto ad una certa lettera di indice
  - Spazio: O(1)

- **func compatibile(schema, parola string) bool**
  Serve a verificare se una data parola è compatibile con uno schema, cioè se può essere una possibile soluzione per quello schema, eseguendo i seguenti controlli:
  1. Lunghezza: Controlla che la parola e lo schema abbiano la stessa lunghezza. Se sono di lunghezza diversa, la funzione restituisce subito false.

2. Corrispondenza lettere fisse: Scorre ogni carattere dello schema e della parola in parallelo. Se nello schema c'è una lettera minuscola, la funzione verifica che nella parola, nella stessa posizione, ci sia esattamente quella stessa lettera. Se non corrisponde, restituisce false.
3. Gestione lettere maiuscole (ripetizioni):
    - Se nello schema c'è una lettera maiuscola, la funzione tiene traccia di quale lettera della parola corrisponde a ogni maiuscola incontrata la prima volta.
    - Se la stessa maiuscola compare più volte nello schema, controlla che nelle stesse posizioni della parola ci sia sempre la stessa lettera già associata.
    - Se si trova una contraddizione (la stessa maiuscola mappata su lettere diverse), restituisce false.

Se tutti i controlli precedenti sono superati, la funzione restituisce true, indicando che la parola è compatibile con lo schema.

Complessità:

- Tempo: O(L) con L lunghezza dello schema passato
- Spazio: O(u) con u <= 26 (caratteri massimi)

- **func (d *Dizionario) ricerca(s string)**

  Cerca tutte le parole nel dizionario che sono compatibili con uno schema S. Per farlo, seleziona solo le parole che hanno la stessa lunghezza dello schema e, per ognuna, controlla se rispetta le regole di compatibilità con lo schema (invocando la funzione compatibile). Infine, stampa lo schema seguito dall'elenco delle parole che lo soddisfano.

  Complessità:

  - Tempo: O(n x L), con n numero di parole di una determinata lunghezza, L complessità della funzione compatibile.
  - Spazio: O(1)

- **func distanza(x, y string) int**

  Calcola la distanza di editing tra due parole x e y basandosi sull'algoritmo di Damerau-Levenshtein. Cioè, il minimo numero di operazioni elementari (inserzione, cancellazione, sostituzione e trasposizione di due caratteri adiacenti) necessarie per trasformare x in y. Procedura:

  1. Gestione dei casi banali: Se una delle due stringhe è vuota, la distanza coincide con la lunghezza dell'altra.
  2. Inizializzazione delle matrici necessarie all'algoritmo: Viene creata una matrice intera d di dimensioni (m+2) × (n+2) dove *m=len(x)* e *n=len(y)*. La variabile *maxDistance := m + n* è un valore sentinella appositamente molto grande. La riga 0 e la colonna 0 valgono *maxDistance*; riga 1 e colonna 1 contengono gli indici standard (da 0 a m e da 0 a n). Ciò facilita il calcolo delle trasposizioni senza uscire dagli indici della matrice.
  3. *da := make(map[rune]int)* è una mappa che memorizza, per ogni carattere già visto in x, l'ultima posizione dove è comparso. All'interno del ciclo esterno, si usa anche db per ricordare l'ultima posizione in y del carattere corrente di x.
  4. Per ogni coppia di valori si applica: sostituzione, inserzione, cancellazione. Si tiene conto per ogniuno di questi il costo ed invocando la funzione min si ricava il costo minimo.
  5. È calcolata anche la trasposizione adiacente, ovvero il costo di scambio di due caratteri consecutivi: *transCost := d[i1][j1] + (i - i1 - 1) + 1 + (j - j1 - 1),* La formula somma:
     - la distanza calcolata fino al precedente incrocio di quei due caratteri (d[i1][j1]).
     - i caratteri intermedi da saltare in x e y.

    - il costo 1 dello scambio.
6. Alla fine dell'iterazione si memorizza *da[rune(x[i-1])] = i* per i prossimi calcoli di trasposizione.
7. Infine, si ritorna *d[m+1][n+1]* che sta ad indicare la distanza finale.

Complessità:

- Tempo: O(m × n) perché si visita ogni cella della matrice una sola volta.
- Spazio: O(m × n) per la matrice numero di righe per numero di colonne.

- **func (d *dizionario) esisteParole(w string) bool**

Verifica se la parola w è presente nel dizionario. Guarda soltanto l'elenco di parole con la stessa lunghezza di w all'interno della mappa indice; se trova la stessa stringa, restituisce true, altrimenti false. Serve quindi come controllo rapido di esistenza prima di operazioni come catena o distanza.

Complessità:

- Tempo: O(n), con n numero di parole
- Spazio: O(1)

- **func (d *dizionario) paroleConDistanza1(w string) []string**

Restituisce tutte le parole del dizionario che distano esattamente 1 operazione di editing da w.

1. Considera solo le parole con lunghezza |w|-1, |w| e |w|+1, perché solo queste possono trovarsi a distanza 1.
2. Per ciascuna di queste parole calcola la distanza invocando *distanza*; se il risultato è 1, la aggiunge alla lista.
3. Ritorna la lista delle parole trovate.

Così genera rapidamente i "vicini" indispensabili alla ricerca della catena minima.

Complessità:

- Tempo: O(n x L^2), con n numero di parole di lunghezza 1, e L la lunghezza di w ed il quadrato dipende dal costo della funzione di distanza su ogni coppia.
- Spazio: O(k + L^2), k numero di vicine trovate, l lunghezza d w.

- **func (d *dizionario) stampaPercorso(x, y string, padre map[string]string)**

Ricostruisce la catena minima da x a y usando la mappa padre (che indica per ogni parola la precedente scoperta nella BFS). Risale da y a x, salva le parole in ordine corretto, poi le stampa.

Complessità:

- Tempo: O(d), con d lunghezza della catena
- Spazio: O(1)

- **func (d *dizionario) catena(x, y string)**

Cerca e stampa una catena di lunghezza minima (ossia il percorso più corto possibile) che trasformi la parola x nella parola y attraverso passi di distanza di editing pari a 1.
Se la catena non esiste, oppure una delle due parole non è contenuta nel dizionario, stampa semplicemente non esiste. Procedura:

1. Controlli preliminari
- Esistenza delle parole: invoca la funzione esisteParola per verificare che x e y siano effettivamente presenti nell'indice per lunghezza; in caso contrario termina subito stampando non esiste.
- Parole uguali: se x == y la catena minima è formata dalla sola parola stessa; la funzione stampa la parola e ritorna.

2. Definizione strutture necessarie per il funzionamento della BFS

- *queue := []string{x}*, contiene le parole da esplorare in ordine di livello.
- *Visited := map[string]bool{x: true}*, assicura che ogni parola venga visitata una sola volta.
- *Padre := map[string]string{}*, memorizza la provenienza di ogni parola scoperta: serve poi a ricostruire il percorso minimo.

3. Ricerca in ampiezza BFS

Finché la coda non è vuota:

- Estrae il primo elemento (current).
- Calcola tutti i vicini a distanza 1 di current invocando la funzione paroleConDistanza1.
    - Quest'ultima esplora soltanto le liste di parole con lunghezza |current|−1, |current| e |current|+1 e usa la funzione distanza per filtrare quelle effettivamente a distanza 1 (riduce in modo significativo il lavoro rispetto a scorrere l'intero dizionario).
- Per ogni vicino non ancora visitato:
    - lo marca come visitato.
    - registra nel dizionario padre che è stato raggiunto da current.
    - se il vicino è y invoca *stampaPercorso* per stampare il cammino ricostruito e termina.
    - altrimenti lo inserisce in coda per l'espansione successiva.

Se la coda si svuota senza aver trovato y, viene stampato "non esiste".

4. Ricostruzione e stampa del percorso

stampaPercorso(x, y, padre) risale la mappa padre partendo da y fino ad arrivare a x, costruendo l'array percorso in ordine inverso; quindi, lo stampa racchiuso tra parentesi tonde, una parola per riga.

Complessità

- Tempo: la BFS visita ogni parola al massimo una volta e genera i vicini con la funzione paroleConDistanza1. Nel caso peggiore: O(V + E) con v = n parole, E = n coppie a distanza 1.
- Spazio: è proporzionale alle strutture queue, visited e padre, cioè al numero di nodi visitati nello scenario peggiore.

5 Algoritmi e procedure alternative

Analizziamo la principale variante all'algoritmo a BFS che avrei potuto implementare, ma che presenta alcune problematiche.

Nel contesto della funzione catena, il requisito fondamentale non è semplicemente raggiungere la parola di arrivo, ma farlo attraversando il minor numero possibile di trasformazioni. Su un grafo non pesato dove i vertici sono le

parole e gli archi uniscono coppie che differiscono di un solo carattere questo equivale a trovare il cammino con meno archi.

La *Breadth-First Search* (BFS) è l'algoritmo che, per costruzione, esplora il grafo "a strati": visita prima tutte le parole a distanza 1, poi quelle a distanza 2 e così via. Di conseguenza, nel nostro caso, la prima volta che incontra la parola di destinazione ha già la certezza di aver scoperto la catena più corta, senza confronti o verifiche aggiuntive. In termini implementativi bastano una coda e una mappa dei padri.

Con la *Depth-First Search* (DFS), invece, il percorso segue un singolo ramo finché possibile per poi retrocedere. Questo comportamento causa due problemi sostanziali:

1. Nessuna garanzia di ottimalità: DFS potrebbe trovare per prima una catena molto lunga, mentre la soluzione ottima si trova in un ramo ancora inesplorato; l'ordine in cui i vicini vengono generati influenza in modo imprevedibile il risultato.
2. Tempi di risposta variabili: in presenza di cicli o di sotto-rami profondi, DFS può impiegare molto più tempo prima di incontrare la meta, e serve codice aggiuntivo per evitare che l'esecuzione entri in un ciclo infinito.

Per queste ragioni, la BFS rappresenta la scelta più razionale: offre **c**ertezza di minimizzare il numero di trasformazioni, tempi di esecuzione prevedibili e un'implementazione lineare che si integra perfettamente con le strutture dati già impiegate nella soluzione proposta.

6 Casi di test e casi limite

	Input	Output
1. Schema con 26 maiuscole tutte distinte	c i abcdefghijklmnopqrstuvwxyz r ABCDEFGHIJKLMNOPQRSTUVWXYZ t	ABCDEFGHIJKLMNOPQRSTUVWXYZ:[abcdefghijklmnopqrstuvwxyz]
2. Dizionario quasi completo di lunghezza 4, ovvero 26^4 parole generando un dizionario estremamente denso. La ricerca tocca centinaia di migliaia di archi.	c i aaaa i aaab … p t	[aaaa aaab aaac ... Zzzz]
3. Catena con lunghezza minima 50, obbligando la BFS a tenere memorizzati molti predecessori e a non fermarsi ad un cammino sub-ottimo	c test50.txt c w0 w50 t	(w0 w1 … w50)
4. Ambiguità di maiuscole ripetute. la funzione compatibile deve applicare la **stessa** lettera a simboli uguali e lettere diverse a simboli diversi, altrimenti produce falsi positivi.	c i abca i acca i abba r ABCA t	ABCA:[abca acca abba]
5. Grafo con cicli fitti che possono portare a cicli infiniti	c i ab i ba i bb i aa c ab aa t	(ab aa)